

DOCUMENTATION TECHNIQUE

Dada Rent Car

Architecture de la base de données, spécifications techniques et automatisations (IA & emailing) de la plateforme web — site public de réservation et panneau d'administration.

Version 1.0 · Généré le 11 septembre 2026

Stack : Angular 22 · Firebase (Auth, Firestore, Storage) · Gemini 3.5 Flash-Lite · Brevo

1. Vue d'ensemble

L'application est un client Angular unique (SPA) qui sert deux expériences distinctes via le routeur :

- **Site public** (/) — vitrine de la flotte, tarifs, et modale de réservation rapide qui enregistre chaque tentative de réservation dans la base de données.
- **Panneau d'administration** (/admin/*) — protégé par authentification, permet à l'équipe Dada de consulter, modifier et suivre chaque réservation, y compris les documents et vidéos associés, avec calcul de prix automatique et notifications par email.

Il n'y a pas de serveur applicatif dédié : Firebase (Google Cloud) sert de backend (base de données temps réel, authentification, stockage de fichiers). Les intégrations tierces (Gemini pour l'IA, Brevo pour l'email) sont appelées directement depuis le navigateur.

2. Architecture applicative

2.1 Structure des dossiers (src/app)

src/app/	
├─ pages/landing/	Page d'accueil publique (assemble les sections marketing)
├─ components/	Sections de la page publique (hero, flotte, tarifs, modale de réservation)
├─ admin/	
│ ├─ login/	Connexion / création de compte admin
│ ├─ shell/	Layout admin (barre du haut, déconnexion)
│ ├─ dashboard/	Liste des réservations, filtrable par statut
│ ├─ attempt-detail/	Fiche complète d'une réservation
│ ├─ attempt-form/	Création manuelle d'une réservation
│ ├─ auth.service.ts	Authentification Firebase
│ └─ auth.guard.ts	Garde de route (accès admin uniquement)
└─ shared/	
│ ├─ firebase.ts	Initialisation Firebase (app, auth, firestore, storage)
│ ├─ attempt.model.ts	Types TypeScript du modèle de données
│ ├─ attempts.service.ts	Accès Firestore/Storage pour les réservations
│ ├─ admins.service.ts	Registre des emails admin (pour les notifications)
│ ├─ pricing.ts	Moteur de calcul de prix (dégressif)
│ ├─ fleet-categories.ts	Catalogue des catégories de véhicules et tarifs
│ ├─ gemini.service.ts	Analyse IA des documents / vidéos
│ └─ email.service.ts	Notifications Brevo

2.2 Routing

Route	Composant	Accès
/	Landing	Public
/admin/login	AdminLogin	Public

/admin	Dashboard	Admin connecté
/admin/new	AttemptForm	Admin connecté
/admin/:id	AttemptDetail	Admin connecté

3. Modèle de données (Cloud Firestore)

Base de données NoSQL orientée documents, organisée en deux collections principales.

3.1 Collection `attempts`

Un document = une tentative / réservation, créée par le site public ou par un admin.

Champ	Type	Description
customerName	string	Nom complet du client
customerPhone	string	Téléphone du client
category	string	Catégorie de véhicule choisie
startDate / endDate	string (AAAA-MM-JJ)	Dates de location
promoCode	string	Code promo optionnel
status	enum	nouveau · confirme · en_cours · termine · annule
source	enum	site ou admin
pricing	objet	{ dailyRate, days, subtotal, discountPct, total }
departureVideos	tableau d'objets	Vidéos d'état des lieux au départ
vehicleDocs	tableau d'objets	Documents véhicule (carte grise...)
clientDocs	tableau d'objets	Documents client (CIN, permis...)
history	tableau d'objets	Journal d'activité { action, note, at }
createdAt / updatedAt	number (epoch ms)	Horodatage

Structure d'un élément de `vehicleDocs` / `clientDocs` / `departureVideos` :

```
{
  url: string,           // URL de téléchargement Firebase Storage
  path: string,          // chemin dans le bucket
}
```

```

name: string,           // nom du fichier original
contentType: string,    // type MIME
uploadedAt: number,
extracted?: {           // documents uniquement – rempli par Gemini
  documentType, fullName, documentNumber, extraInfo
},
plateInfo?: {           // vidéos de départ uniquement – rempli par Gemini
  plateNumber, vehicleCondition, notes
}
}

```

3.2 Collection `admins`

Un document par administrateur (id = uid Firebase Auth), utilisé pour la liste de diffusion des notifications email.

Champ	Type	Description
<code>email</code>	string	Email de l'administrateur
<code>createdAt</code>	number	Date d'inscription

4. Firebase Storage

Fichiers organisés par réservation : `attempts/{id}/{kind}/{timestamp}-{nom_fichier}`, où `{kind}` vaut `departureVideos`, `vehicleDocs` ou `clientDocs`.

5. Authentification & sécurité

5.1 Authentification

Firebase Authentication (email / mot de passe). La création de compte admin est en libre-service depuis la page de connexion ; chaque inscription enregistre automatiquement l'email dans la collection `admins` pour les notifications.

5.2 Règles d'accès (Firestore & Storage)

Collection / chemin	Création	Lecture	Modification / suppression
<code>attempts</code>	Public (formulaire de réservation)	Admin connecté	Admin connecté
<code>admins</code>	Soi-même uniquement	Admin connecté	Soi-même uniquement
Storage <code>attempts/**</code>	Admin connecté (lecture & écriture)		—

État actuel : les fichiers `firestore.rules` et `storage.rules` du dépôt définissent ce comportement, mais doivent être publiés manuellement dans la console Firebase (Firestore → Règles / Storage → Règles)

pour être actifs. Tant qu'ils ne le sont pas, le projet reste en mode ouvert (lecture/écriture non restreinte) — à corriger avant la mise en production.

6. Automatisation IA — Gemini 3.5 Flash-Lite

Utilisé exclusivement pour la **lecture structurée de documents et vidéos** — jamais pour les calculs de prix, qui restent du code déterministe (voir section 8) afin d'éviter tout risque d'erreur de calcul généré par IA.

6.1 Documents (client & véhicule)

À l'ajout d'un document (carte d'identité, permis, carte grise), l'image/PDF est envoyée au modèle avec une consigne de réponse strictement JSON :

```
{ "documentType": "...", "fullName": "...", "documentNumber": "...", "extraInfo": "..." }
```

6.2 Vidéos de départ — lecture de la matricule

À l'ajout d'une vidéo d'état des lieux, Gemini analyse la vidéo pour repérer la plaque d'immatriculation et l'état visible du véhicule :

```
{ "plateNumber": "...", "vehicleCondition": "...", "notes": "..." }
```

Une limite de taille (~18 Mo) évite d'envoyer des vidéos trop volumineuses en base64 — au-delà, l'analyse automatique est simplement ignorée sans bloquer l'upload.

Le modèle utilisé est **gemini-3.5-flash-lite**, appelé via l'endpoint REST `generateContent` de l'API Gemini, avec `responseMimeType: application/json` pour garantir une sortie strictement structurée et exploitable directement par l'interface.

7. Automatisation Email — Brevo

Notification automatique par email à chaque événement clé d'une réservation :

Déclencheur	Quand
Nouvelle réservation	Chaque création (site public ou admin)
Véhicule échangé	Changement de catégorie sur une réservation existante
Dates modifiées (prolongation)	Changement des dates de départ/retour
Réservation annulée	Passage au statut "Annulé"

Destinataires : une adresse fixe (propriétaire) + la liste complète des emails de la collection `admins` (déduplication automatique). Chaque email contient un résumé (client, véhicule, dates, prix total, statut) et un lien direct vers la fiche dans le panneau admin.

8. Moteur de calcul des prix

Calcul entièrement déterministe (aucune IA), basé sur le tarif journalier de la catégorie et la durée de location :

Durée	Réduction appliquée
1 à 6 jours	Aucune (tarif standard)
7 à 29 jours	-8 %
30 jours et plus	-12 %

`total = tarif_journalier × nombre_de_jours × (1 - réduction)` , recalculé automatiquement à chaque changement de véhicule ou de dates.

9. Actions disponibles sur une réservation

- **Changement de statut** — 5 statuts représentant le cycle de vie complet.
- **Échange de véhicule** — changement de catégorie avec recalcul de prix immédiat.
- **Prolongation** — modification des dates (raccourcis +1 / +7 / +30 jours) avec recalcul de prix.
- **Annulation** — action dédiée avec confirmation, distincte des autres statuts.
- **Ajout de justificatifs** — vidéos de départ, documents véhicule, documents client, avec lecture automatique par IA.

10. Récapitulatif technique

Composant	Technologie
Framework front-end	Angular 22 (composants standalone, signals, nouveau control-flow <code>@if</code> / <code>@for</code>)
Langage	TypeScript
Build / dev server	Angular CLI (esbuild + Vite)
Base de données	Cloud Firestore (temps réel)
Authentification	Firebase Authentication (email / mot de passe)
Stockage fichiers	Firebase Storage

IA / analyse documentaire	Gemini 3.5 Flash-Lite (Google)
Emailing transactionnel	Brevo (API REST)
Tests unitaires	Vitest

Points de vigilance avant mise en production :

- Les clés API Gemini et Brevo sont actuellement intégrées côté client (visibles dans le code source du navigateur). À terme, les déplacer derrière une fonction serveur (Cloud Functions) pour éviter toute utilisation abusive.
- Publier les règles `firestore.rules` / `storage.rules` dans la console Firebase (actuellement non appliquées).
- Activer le fournisseur "Email/Mot de passe" dans Firebase Authentication si ce n'est pas déjà fait.